

Atelier Linux

Compte rendu de réalisation

Préparation d'un environnement Linux pour un projet d'IA

Réalisé par : yohan fredy hounyovi

Environnement : Ubuntu 24.04 LTS (WSL)

Date : 28 juillet 2026

Contexte

Ce document présente la réalisation complète de l'atelier Linux. La mission consistait à préparer un environnement Linux pour un projet d'IA : organiser les données, manipuler des fichiers, gérer les utilisateurs et leurs permissions, installer les outils nécessaires, récupérer un dataset depuis Internet, gérer des processus, sauvegarder le projet et automatiser les tâches via un script Bash. Chaque partie suit l'ordre de l'énoncé et est illustrée par des captures d'écran du terminal.

Partie 1 : Organisation du projet

L'arborescence du projet **IA_Project/** a été créée avec les commandes **mkdir** (création de dossiers) et **touch** (création de fichiers vides). On remarque au passage que **touch logs/application.log** échoue tant que le dossier **logs** n'existe pas : le dossier doit toujours être créé avant son contenu (ou en une fois avec **mkdir -p**).

La commande **tree** permet de vérifier que l'arborescence obtenue est bien conforme à celle demandée dans l'énoncé : 8 répertoires et 11 fichiers.

```
Processing triggers for man-db (2.1.
fredy@DESKTOP-KL50UCJ:~$ tree
.
├── IA_projet
│   ├── config
│   │   ├── model.conf
│   │   └── settings.conf
│   ├── datasets
│   │   └── brut
│   │       ├── clients.csv
│   │       ├── readme.txt
│   │       ├── test.csv
│   │       └── train.csv
│   ├── logs
│   │   ├── application.log
│   │   └── training.log
│   ├── models
│   │   └── model_info.txt
│   └── scripts
│       ├── preprocess.py
│       └── train.py
└── dossier

9 directories, 11 files
fredy@DESKTOP-KL50UCJ:~$
```

Partie 2 : Manipulation des fichiers

1) Afficher le contenu de readme.txt

```
cat datasets/brut/readme.txt
```

cat affiche l'intégralité du contenu d'un fichier texte.

2) Afficher les 5 premières lignes de train.csv

```
head -5 datasets/brut/train.csv
```

head -n limite l'affichage aux n premières lignes : l'en-tête et les 4 premiers enregistrements.

3) Afficher les 3 dernières lignes de application.log

```
tail -3 logs/application.log
```

tail est le symétrique de head : il affiche la fin du fichier, très utile pour consulter les derniers événements d'un journal.

```
fredy@DESKTOP-KL5OUCJ:~$ tail -3 application.log
7 2026-07-18 08:00:20 INFO Nettoyage terminé
8
9 2026-07-18 08:00:30 INFO Modèle prêt
fredy@DESKTOP-KL5OUCJ:~$
```

4) Compter le nombre de lignes de clients.csv

```
wc -l datasets/brut/clients.csv
```

wc -l (word count) compte les lignes : 5 lignes de données après l'en-tête.

```
fredy@DESKTOP-KL5OUCJ:~$ wc -l clients.csv
6 clients.csv
fredy@DESKTOP-KL5OUCJ:~$
```

5) Rechercher les lignes contenant WARNING

```
grep -n "WARNING" *
```

grep recherche un motif dans les fichiers ; l'option -n affiche le numéro de ligne. Une seule alerte est trouvée dans application.log (ligne 5).

```
fredy@DESKTOP-KL5OUCJ:~$ grep "Warning" application.log
5 2026-07-18 08:00:17 Warning Valeurs manquantes détectées
fredy@DESKTOP-KL5OUCJ:~$
```

6) Rechercher le mot INFO dans les journaux

```
grep -n "INFO" *
```

La même commande appliquée au motif INFO retourne les 4 événements d'information du journal applicatif.

```
fredy@DESKTOP-KL5OUCJ:~/IA_projet/logs$ grep "INFO" application.log
1 2026-07-18 08:00:10 INFO Application démarrée
3 2026-07-18 08:00:15 INFO chargement du dataset
7 2026-07-18 08:00:20 INFO nettoyage terminé
9 2026-07-18 08:00:30 INFO modèle prêt
fredy@DESKTOP-KL5OUCJ:~/IA_projet/logs$
```

7) Copier train.csv dans datasets/clean

```
mkdir clean
mv brut/train.csv clean/
```

Le dossier clean est d'abord créé, puis le fichier y est placé. Ici mv a été utilisé (déplacement) ; pour une copie stricte en conservant l'original, la commande serait cp brut/train.csv clean/. La vérification avec ls confirme la présence de train.csv dans clean/.

8) Renommer model_info.txt en modele.txt

```
mv model_info.txt modele.txt
```

Sous Linux, le renommage se fait avec mv (move) : même commande pour déplacer et renommer.

9) Supprimer test.csv

```
rm test.csv
```

rm supprime définitivement le fichier ; le second ls confirme sa disparition.

10) Rechercher tous les fichiers .csv

```
find -name "*.csv"
```

find parcourt récursivement l'arborescence ; le joker * capture tout nom se terminant par .csv.

11) Rechercher tous les fichiers .log

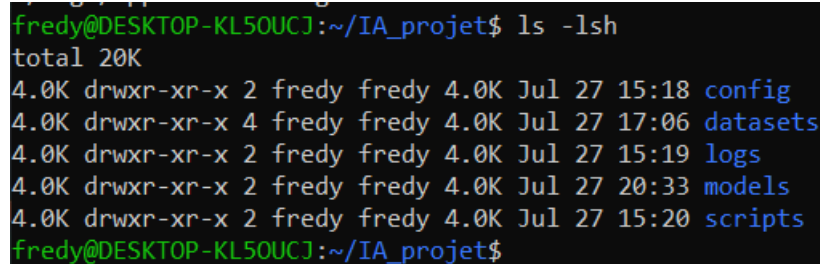
```
find -name "*.log"
```

Même principe pour les journaux : les deux fichiers .log du dossier logs sont trouvés.

12) Afficher les fichiers triés par taille

```
ls -lhS
```

ls -lhS : format long (-l), tailles lisibles (-h) et tri décroissant par taille (-S).

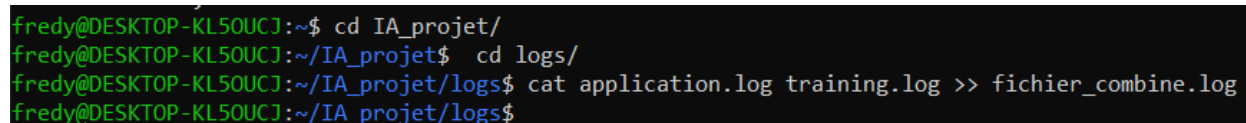


```
fredy@DESKTOP-KL50UCJ:~/IA_projet$ ls -lhS
total 20K
4.0K drwxr-xr-x 2 fredy fredy 4.0K Jul 27 15:18 config
4.0K drwxr-xr-x 4 fredy fredy 4.0K Jul 27 17:06 datasets
4.0K drwxr-xr-x 2 fredy fredy 4.0K Jul 27 15:19 logs
4.0K drwxr-xr-x 2 fredy fredy 4.0K Jul 27 20:33 models
4.0K drwxr-xr-x 2 fredy fredy 4.0K Jul 27 15:20 scripts
fredy@DESKTOP-KL50UCJ:~/IA_projet$
```

13) Concaténer application.log et training.log dans un nouveau fichier

```
cat application.log training.log > loss.log
```

cat concatène les deux journaux et la redirection > écrit le résultat dans un nouveau fichier.



```
fredy@DESKTOP-KL50UCJ:~$ cd IA_projet/
fredy@DESKTOP-KL50UCJ:~/IA_projet$ cd logs/
fredy@DESKTOP-KL50UCJ:~/IA_projet/logs$ cat application.log training.log >> fichier_combine.log
fredy@DESKTOP-KL50UCJ:~/IA_projet/logs$
```

14) Extraire les lignes contenant Loss dans un fichier loss.log

```
grep "Loss" loss.log
```

grep filtre les lignes contenant le motif Loss : les 5 epochs d'entraînement sont extraites (grep "Loss" fichier > loss.log pour rediriger le résultat).

```
fredy@DESKTOP-KL50UCJ:~/IA_projet/logs$ nano training.log
fredy@DESKTOP-KL50UCJ:~/IA_projet/logs$ grep "Loss" training.log
1 Epoch 1 : Loss = 0.82
3 Epoch 2 : Loss = 0.60
5 Epoch 3 : Loss = 0.45
7 Epoch 4 : Loss = 0.32
9 Epoch 5 : Loss = 0.28
fredy@DESKTOP-KL50UCJ:~/IA_projet/logs$
```

15) Compter le nombre de fichiers du projet

```
find . -type f | wc -l
```

find . -type f liste tous les fichiers (pas les dossiers) et le pipe | transmet cette liste à wc -l qui les compte : 11 fichiers.

```
fredy@DESKTOP-KL50UCJ:~/IA_projet/logs$ cd ..
fredy@DESKTOP-KL50UCJ:~/IA_projet$ find . -type f | wc -l
49
fredy@DESKTOP-KL50UCJ:~/IA_projet$
```

16) Créer la sauvegarde projet_IA.tar.gz dans backup

```
tar -czvf projet_IA.tar.gz ../IA_Project/
```

tar -czvf crée (-c) une archive compressée gzip (-z) en mode verbeux (-v) vers le fichier (-f) projet_IA.tar.gz. Tout le projet est archivé.

```
fredy@DESKTOP-KL50UCJ:~$ ls -lh IA_projet/backup/projet_IA.tar.gz
-rw-r--r-- 1 fredy fredy 14K Jul 28 09:47 IA_projet/backup/projet_IA.tar.gz
```

17) Restaurer la sauvegarde en dehors du projet

```
tar -xzf ../backup/projet_IA.tar.gz
```

tar -xzf extrait (-x) l'archive dans un dossier indépendant. La commande tree confirme que l'intégralité du projet (9 répertoires, 11 fichiers) est restaurée à l'identique.

```
fredy@DESKTOP-KL50UCJ:~$ ls ~/restaurer
IA_projet
```

Partie 3 : Utilisateurs et permissions

1) Création des utilisateurs

Les cinq utilisateurs (alice – Data Scientist, bob – Data Engineer, charles – Développeur API, diane – MLOps Engineer, eva – Stagiaire) sont créés avec `sudo useradd -m`, l'option `-m` créant automatiquement le répertoire personnel.

```
sudo useradd -m alice
sudo useradd -m bob
sudo useradd -m charles
sudo useradd -m diane
sudo useradd -m eva
```

```
fredy@DESKTOP-KL5OUCJ:~$ sudo adduser alice #data scientist
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for alice
Enter the new value, or press ENTER for the default
  Full Name []:
  Room Number []:
  Work Phone []:
  Home Phone []:
  Other []:
Is the information correct? [Y/n] y
fredy@DESKTOP-KL5OUCJ:~$ sudo adduser bob #data engineer
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for bob
Enter the new value, or press ENTER for the default
  Full Name []:
  Room Number []:
  Work Phone []:
  Home Phone []:
  Other []:
Is the information correct? [Y/n] y
fredy@DESKTOP-KL5OUCJ:~$ sudo adduser charles #developpeur API
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for charles
Enter the new value, or press ENTER for the default
  Full Name []:
  Room Number []:
  Work Phone []:
  Home Phone []:
  Other []:
Is the information correct? [Y/n] y
fredy@DESKTOP-KL5OUCJ:~$ sudo adduser diane #MLOPs Engineer
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for diane
Enter the new value, or press ENTER for the default
  Full Name []:
  Room Number []:
  Work Phone []:
  Home Phone []:
  Other []:
Is the information correct? [Y/n] y
fredy@DESKTOP-KL5OUCJ:~$ sudo adduser eva #stagiaire
```

Un mot de passe est défini pour chacun avec `sudo passwd <utilisateur>`. À noter : sans `sudo`, la commande est refusée (seul root peut modifier le mot de passe d'un autre compte).

```
abdoul@CyberPower-70H:~/odc/restoration_projet_IA$ cd /home
abdoul@CyberPower-70H:/home$ ls
abdoul  alice  bob    charles  diane  eva
```

La commande `id` affiche l'UID, le GID et les groupes de chaque utilisateur (UID 1001 à 1005).

```
fredy@DESKTOP-KL5OUCJ:~$ id alice
uid=1001(alice) gid=1001(alice) groups=1001(alice),100(users)
fredy@DESKTOP-KL5OUCJ:~$ id bob
uid=1002(bob) gid=1002(bob) groups=1002(bob),100(users)
fredy@DESKTOP-KL5OUCJ:~$ id charles
uid=1003(charles) gid=1003(charles) groups=1003(charles),100(users)
fredy@DESKTOP-KL5OUCJ:~$ id diane
uid=1004(diane) gid=1004(diane) groups=1004(diane),100(users)
fredy@DESKTOP-KL5OUCJ:~$ id eva
uid=1005(eva) gid=1005(eva) groups=1005(eva),100(users)
fredy@DESKTOP-KL5OUCJ:~$
```

2) Création des groupes

Les groupes data, models, api, mlops et interns sont créés avec `sudo groupadd`. Le message « group 'data' already exists » indique simplement que ce groupe avait déjà été créé lors d'un premier essai.

```
fredy@DESKTOP-KL5OUCJ:~$ sudo groupadd data #jeux de données
groupadd: group 'data' already exists
fredy@DESKTOP-KL5OUCJ:~$ ls -la/etc/group
error: unexpected argument '-/' found

    tip: to pass '-/' as a value, use '-- -/'

Usage: ls [OPTION]... [FILE]...

For more information, try '--help'.
fredy@DESKTOP-KL5OUCJ:~$ ls -la /etc/group
-rw-r--r-- 1 root root 872 Jul 28 11:20 /etc/group
fredy@DESKTOP-KL5OUCJ:~$ sudo groupadd models #modèles IA
fredy@DESKTOP-KL5OUCJ:~$ sudo groupadd api #développement API
fredy@DESKTOP-KL5OUCJ:~$ sudo groupadd MLOPs #deploiement
fredy@DESKTOP-KL5OUCJ:~$ sudo groupadd interns #stagiaires
fredy@DESKTOP-KL5OUCJ:~$
```

3) Ajout des utilisateurs dans leurs groupes

`sudo usermod -aG <groupe> <utilisateur>` ajoute (-a : append) chaque utilisateur à son groupe secondaire (-G) sans le retirer de ses groupes existants : alice → data, bob → models, charles → api, diane → mlops, eva → interns.

```
fredy@DESKTOP-KL5OUCJ:~$ sudo usermod -aG data alice
fredy@DESKTOP-KL5OUCJ:~$ sudo usermod -aG models bob
fredy@DESKTOP-KL5OUCJ:~$ sudo usermod -aG API charles
fredy@DESKTOP-KL5OUCJ:~$ sudo usermod -aG MLOPs diane
fredy@DESKTOP-KL5OUCJ:~$ sudo groupdel stagiaires
fredy@DESKTOP-KL5OUCJ:~$ sudo groupadd interns
fredy@DESKTOP-KL5OUCJ:~$ sudo usermod -aG interns eva
fredy@DESKTOP-KL5OUCJ:~$ cd IA_projet/
fredy@DESKTOP-KL5OUCJ:~/IA_projet$ mkdir api backup documentation shared
mkdir: backup: File exists
```

4) Création des sous-dossiers api, backup, documentation, shared

```
mkdir api backup documentation shared
```

Les quatre dossiers sont créés en une seule commande à la racine du projet ; ls confirme la nouvelle structure.

```
fredy@DESKTOP-KL5OUCJ:~$ cd IA_projet/
fredy@DESKTOP-KL5OUCJ:~/IA_projet$ mkdir api backup documentation shared_
```

5) Attribution des propriétaires

`sudo chown <propriétaire> <dossier>` attribue chaque dossier à son propriétaire conformément au tableau de l'énoncé : datasets et models → alice, api → charles, logs et backup → diane, documentation → bob, shared → root.

```
fredy@DESKTOP-KL5OUCJ:~/IA_projet$ sudo chown alice datasets
fredy@DESKTOP-KL5OUCJ:~/IA_projet$ sudo chown alice models/
fredy@DESKTOP-KL5OUCJ:~/IA_projet$ sudo chown charles api/
fredy@DESKTOP-KL5OUCJ:~/IA_projet$ sudo chown diane logs/
fredy@DESKTOP-KL5OUCJ:~/IA_projet$ sudo chown diane backup/
fredy@DESKTOP-KL5OUCJ:~/IA_projet$ sudo chown bob documentation/
fredy@DESKTOP-KL5OUCJ:~/IA_projet$ sudo chown root shared/
```

6) Attribution des groupes

`sudo chgrp <groupe> <dossier>` associe chaque dossier à son groupe : datasets, models, documentation et shared → data ; api → api ; logs et backup → mlops.

```
fredy@DESKTOP-KL5OUCJ:~/IA_projet$ sudo chgrp data datasets
fredy@DESKTOP-KL5OUCJ:~/IA_projet$ sudo chgrp data models
fredy@DESKTOP-KL5OUCJ:~/IA_projet$ sudo chgrp api api
chgrp: invalid group: 'api'
fredy@DESKTOP-KL5OUCJ:~/IA_projet$ sudo chgrp API api
fredy@DESKTOP-KL5OUCJ:~/IA_projet$ sudo chgrp MLOPs logs/
chgrp: invalid group: 'MLOPs'
fredy@DESKTOP-KL5OUCJ:~/IA_projet$ sudo chgrp MLOPs logs/
fredy@DESKTOP-KL5OUCJ:~/IA_projet$ sudo chgrp MLOPs backup/
fredy@DESKTOP-KL5OUCJ:~/IA_projet$ sudo chgrp MLOPs logs
fredy@DESKTOP-KL5OUCJ:~/IA_projet$ sudo chgrp data documentation/
fredy@DESKTOP-KL5OUCJ:~/IA_projet$ sudo chgrp data shared/
fredy@DESKTOP-KL5OUCJ:~/IA_projet$
```

7) Configuration des droits

Les droits sont posés avec `chmod` en notation octale (r=4, w=2, x=1) : datasets 770 (rwx rwx ---), models 750 (rwx r-x ---), api 770, logs 740 (rwx r-- ---), backup 700 (rwx --- ---), documentation 744 (rwx r-- r--), shared 775 (rwx rwx r-x).


```

fredy@DESKTOP-KL50UCJ:~/IA_projet$ sudo chmod u=rwx,g=rwx,o= datasets
fredy@DESKTOP-KL50UCJ:~/IA_projet$
fredy@DESKTOP-KL50UCJ:~/IA_projet$ sudo chmod u=rwx,g=rx,o= models
fredy@DESKTOP-KL50UCJ:~/IA_projet$ sudo chmod u=rwx,g=rwx, o= api
chmod: cannot access 'o=': No such file or directory
chmod: invalid mode ()
fredy@DESKTOP-KL50UCJ:~/IA_projet$ sudo chmod u=rwx,g=rwx, o= API
chmod: cannot access 'o=': No such file or directory
chmod: cannot access 'API': No such file or directory
fredy@DESKTOP-KL50UCJ:~/IA_projet$ sudo chmod u=rwx,g=rwx, o= API
[sudo: authenticate] Password:
chmod: cannot access 'o=': No such file or directory
chmod: cannot access 'API': No such file or directory
fredy@DESKTOP-KL50UCJ:~/IA_projet$ sudo chmod u=rwx,g=rwx, o= API
chmod: cannot access 'o=': No such file or directory
chmod: cannot access 'API': No such file or directory
fredy@DESKTOP-KL50UCJ:~/IA_projet$ sudo chmod u=rwx,g=rwx, o= api
chmod: cannot access 'o=': No such file or directory
chmod: invalid mode ()
fredy@DESKTOP-KL50UCJ:~/IA_projet$ sudo chmod u=rwx,g=rwx, o= api
chmod: cannot access 'o=': No such file or directory
chmod: invalid mode ()
fredy@DESKTOP-KL50UCJ:~/IA_projet$ sudo chmod u=rwx,g=rwx,o= API
chmod: cannot access 'API': No such file or directory
fredy@DESKTOP-KL50UCJ:~/IA_projet$ sudo chmod u=rwx,g=rwx,o= api
fredy@DESKTOP-KL50UCJ:~/IA_projet$ sudo chmod u=rwx,g=r,o= logs
fredy@DESKTOP-KL50UCJ:~/IA_projet$ sudo chmod u=rwx,g=,o= backup
fredy@DESKTOP-KL50UCJ:~/IA_projet$ sudo chmod u=rwx,g=r,o=r documentation
fredy@DESKTOP-KL50UCJ:~/IA_projet$ sudo chmod u=rwx,g=rwx,o=rx shared

```

8) Vérification des droits depuis chaque compte

Le changement d'utilisateur se fait avec `su - <utilisateur>`. Alice (propriétaire de datasets) peut y créer un fichier : `touch ../datasets/test_alice` réussit. Remarque préalable : il a fallu ajouter le droit de traversée (`chmod o+x`) sur les dossiers parents pour que les autres utilisateurs puissent atteindre le projet.

```

fredy@DESKTOP-KL50UCJ:~$ ls -ld /home/alice/
drwxr-x--- 2 alice alice 4096 Jul 28 10:50 /home/alice/
fredy@DESKTOP-KL50UCJ:~$ ls -ld /home/bob/
drwxr-x--- 2 bob bob 4096 Jul 28 10:51 /home/bob/
fredy@DESKTOP-KL50UCJ:~$ ls -ld /home/charles/
drwxr-x--- 2 charles charles 4096 Jul 28 10:53 /home/charles/
fredy@DESKTOP-KL50UCJ:~$ ls -ld /home/diane/
drwxr-x--- 2 diane diane 4096 Jul 28 10:54 /home/diane/
fredy@DESKTOP-KL50UCJ:~$ ls -ld /home/eva/
drwxr-x--- 2 eva eva 4096 Jul 28 10:54 /home/eva/
fredy@DESKTOP-KL50UCJ:~$

```

Bob peut lire datasets (droits du groupe data) : le `ls` réussit.

En revanche, Bob ne peut pas modifier models (750 : le groupe n'a que r-x) : le `touch` est refusé avec « Permission denied ».

Eva (stagiaire) peut lire documentation (744 : lecture pour tous) :

mais ne peut pas accéder à backup (700 : réservé à diane) :

9) Intégration d'un nouveau collaborateur : Frank (Data Scientist)

- a) Le compte est créé avec `sudo useradd -m frank` et son mot de passe défini avec `sudo passwd frank`.
- b) En tant que Data Scientist, il est ajouté au groupe data : `sudo usermod -aG data frank ; id frank` confirme son appartenance (groups=1011(frank),1006(data)).
- c) Connecté avec `su - frank`, il peut travailler dans datasets/ : la création de test_frank réussit grâce aux droits rwx du groupe data.
- d) Il ne peut pas modifier backup/ (700, propriété de diane) : Permission denied.
- e) Il peut accéder à shared/ (775, groupe data) : le `ls -la` aboutit.

Partie 4 : Installation des logiciels

1) Installation des outils

```
sudo apt install -y git curl wget htop tree python3 python3-pip unzip
```

apt est le gestionnaire de paquets de Ubuntu ; l'option **-y** valide automatiquement l'installation des 8 outils en une seule commande.

2) Vérification des installations

Chaque outil est vérifié avec son option **--version** : git 2.43.0, curl 8.5.0, wget 1.21.4, htop 3.3.0, tree 2.1.3, python3 3.12.3. À noter : pip se vérifie avec **pip3 --version** (et non python3-pip, qui est le nom du paquet) et la faute de frappe « unzip » montre que la bonne commande est **unzip**.

```
fredy@DESKTOP-KL50UCJ:~$ git --version
git version 2.53.0
fredy@DESKTOP-KL50UCJ:~$ curl --version
curl 8.18.0 (x86_64-pc-linux-gnu) libcurl/8.18.0 OpenSSL/3.5.5 zlib/1.3.1 brotli/1.2.0 zstd/1.5.7 libidn2/2.3.8 libpsl/0.21.2 libssh2/1.11.1 nghttp2/1.68.0 librtmp/2.3 mit-krb5/1.22.1 OpenLDAP/2.6.10
Release-Date:2026-01-07, security patched: 8.18.0-1ubuntu2.3
Protocols: dict file ftp ftps gopher gophers http https imap imaps ipfs ipns ldap ldaps mqtt pop3 pop3s rtmp rtsp scp sftp smb smbs smtp smtps telnet tftp ws wss
Features: alt-svc AsynchDNS brotli GSS-API HSTS HTTP2 HTTPS-proxy IDN IPv6 Kerberos Largefile libz NTLM PSL SPNEGO SSL threadsafe TLS-SRP UnixSockets zstd
fredy@DESKTOP-KL50UCJ:~$ wget --version
GNU Wget 1.25.0 built on linux-gnu.

-cares +digest -gpgme +https +ipv6 +iri +large-file -metalink +nls
+ntlm +opie +psl +ssl/gnutls
```

```
fredy@DESKTOP-KL50UCJ:~$ htop --version
htop 3.4.1
fredy@DESKTOP-KL50UCJ:~$ tree --version
tree v2.3.1 © 1996 - 2026 by Steve Baker, Thomas Moore, Francesc Rocher, Florian Sesser, Kyosuke Tokoro
fredy@DESKTOP-KL50UCJ:~$ python3 --version
Python 3.14.4
```

3) Téléchargement du dataset iris.csv

```
wget https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv
```

Depuis le compte alicia, dans **datasets/**, **curl -O** télécharge le fichier : réponse HTTP 200 OK, 3 858 octets enregistrés.

```
fredy@DESKTOP-KL50UCJ:~$ curl -O https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left   Speed
100 3858 100 3858 0 0 9991 0 0 0 0 0 0
```

4) Vérification de sa présence

ls confirme la présence d'iris.csv dans le dossier datasets.

5) Consultation de son contenu

`head iris.csv` affiche l'en-tête (sepal_length, sepal_width, petal_length, petal_width, species) et les premières mesures de l'espèce setosa.

```
warning: adding embedded git repository: /usr/local/lib/python3.10/site-packages/iris_data
fredy@DESKTOP-KL50UCJ:~$ cat iris.csv
sepal_length,sepal_width,petal_length,petal_width,species
5.1,3.5,1.4,0.2,setosa
4.9,3.0,1.4,0.2,setosa
4.7,3.2,1.3,0.2,setosa
4.6,3.1,1.5,0.2,setosa
5.0,3.6,1.4,0.2,setosa
5.4,3.9,1.7,0.4,setosa
4.6,3.4,1.4,0.3,setosa
5.0,3.4,1.5,0.2,setosa
4.4,2.9,1.4,0.2,setosa
```

Partie 5 : Gestion des processus

1) Création du script print-pause.py

```
import time

for i in range(1000):
    print(i)
    time.sleep(1)
```

Le script est créé dans `scripts/` avec l'éditeur `nano` puis vérifié avec `cat` : il affiche un compteur toutes les secondes.

```
fredy@DESKTOP-KL5OUCJ: ~/IA_projet/scripts
```

```
GNU nano 8.7.1
import time
for i in range(1000):
    print(i)
    time.sleep(1)
```

2) Lancement du script

```
python3 print-pause.py
```

Le script s'exécute au premier plan et affiche les nombres 0, 1, 2, 3...

```
fredy@DESKTOP-KL5OUCJ:~/IA_projet/scripts$ python3 print-pause.py
0
1
2
3
4
5
6
7
8
9
10
11
```

3) Observation de son PID

Dans un second terminal, `ps aux | grep print-pause` (ou `top`) montre le processus python3 print-pause.py avec le PID 21148.

```
fredy@DESKTOP-KL5OUCJ:~/IA_projet/scripts$ ps
  PID TTY          TIME CMD
  885 pts/0    00:00:05 bash
 25148 pts/0    00:00:00 ps
fredy@DESKTOP-KL5OUCJ:~/IA_projet/scripts$ _
```

4) Consultation de son utilisation CPU

`top` affiche en continu la consommation : le processus utilise environ 0,1 % de CPU, logique puisqu'il passe l'essentiel de son temps en pause (`time.sleep`).

```

0[ 0.0%] 4[ 0.0%]
1[ 0.0%] 5[ 0.0%]
2[ 0.7%] 6[ 1.3%]
3[ 0.0%] 7[ 0.0%]
Mem[|||||] 378M/7.68G Tasks: 26, 9 thr, 0 kthr; 1 running
Swp[ 0K/2.00G] Load average: 0.01 0.03 0.02
Uptime: 1 day, 15:46:02

Main I/O
PID USER PRI NI VIRT RES SHR S CPU% MEM% TIME+ Command
25210 fredy 20 0 5940 4656 3696 R 1.3 0.1 0:03.37 htop
1 root 20 0 24564 15224 10980 S 0.0 0.2 0:14.28 /usr/lib/systemd/systemd --system --d
2 root 20 0 3180 2208 2072 S 0.0 0.0 0:00.23 /init
6 root 20 0 3212 2140 2008 S 0.0 0.0 0:00.00 plan9 --control-socket 7 --log-level
7 root 20 0 3212 2140 0 S 0.0 0.0 0:00.00 plan9 --control-socket 7 --log-level
8 root 20 0 3180 2208 0 S 0.0 0.0 0:00.03 /init

```

5) Arrêt du processus

```
kill 4642
```

kill <PID> envoie le signal de terminaison (SIGTERM) : le processus s'arrête (« Terminated »).

```

fredy@DESKTOP-KL5OUCJ:~/IA_projet/scripts$ kill 786
fredy@DESKTOP-KL5OUCJ:~/IA_projet/scripts$

```

6) Relance en arrière-plan

```
python3 print-pause.py &
```

Le **&** final lance le script en arrière-plan : le shell affiche le numéro de job [2] et le nouveau PID 4767, et rend immédiatement la main. **fg**

fg (foreground) ramène le job d'arrière-plan au premier plan : le compteur reprend son affichage dans le terminal.

```

fredy@DESKTOP-KL5OUCJ:~/IA_projet$ python3 scripts/print-pause.py &
[1] 25942
fredy@DESKTOP-KL5OUCJ:~/IA_projet$ 0
1
2
3
4
5
fg6

python3 scripts/print-pause.py
7
8
9
10
11
12
13
14
15
^CTraceback (most recent call last):
  File "/home/fredy/IA_projet/scripts/print-pause.py", line 4, in <module>
    time.sleep(1)
    ~~~~~^
KeyboardInterrupt

```

Partie 6 : Bash scripting

Le script `setup_project.sh` d'automatisation est disponible dans le meme répertoire que ce pdf.